OPERATING SYSTEMS

ASSIGNMENT 1 – CO1

Evaluate and Justify the Operating-System Design of a Smart Hospital

Submitted by

MOHAMMED FAREETH S A

Register Number: 192511288

Course: CSA0417 – Operating Systems

Executive Summary

A single general-purpose operating system should not be deployed on every hospital device. The safest design is a small, deterministic real-time operating system on bedside monitors; a secure handheld operating system on tablets; a managed desktop operating system on workstations; a highly available clustered database service; a multiprocessor Linux imaging server using DMA-capable I/O; and a distributed, encrypted network connection between hospitals. The design should be modified—not discarded—when home monitoring and cloud records are added.

- Patient safety has priority: monitor alarms must meet a verified 10 ms deadline.
- Availability and data integrity dominate the patient-record database design.
- Throughput and parallel processing dominate MRI image processing.
- Battery life, secure mobility and manageable updates dominate handheld tablets.
- Network sharing must use authentication, encryption, access control, auditing and recovery.

TASK 1 – Evaluate the Requirements of Each Computing Environment

1.1 Ranked requirements

Environment	Ranked requirements	Justification
Bedside patient monitors	1. Deterministic response (≤10 ms); 2. Reliability; 3. Sensor I/O accuracy; 4. Security; 5. Low memory/power use	A delayed alarm can directly endanger a patient. Predictable worst-case response is more important than general throughput.
Nurses' tablets	1. Battery life; 2. Security/privacy; 3. Reliable wireless communication; 4. Usability; 5. Responsiveness	Tablets are mobile and used throughout a shift. They need power management and controlled access to patient data.
Doctors' workstations	1. Security; 2. Responsiveness; 3. Reliability; 4. Compatibility; 5. Multi-user/network access	They run clinical applications and display records and images. A brief delay is tolerable, but confidentiality and availability are essential.
Patient-record database	1. Availability; 2. Data integrity; 3. Security; 4. Transaction throughput; 5. Recoverability	An outage or corrupted record can affect clinical decisions. Redundancy, backups and transaction control are essential.
MRI imaging server	1. High throughput; 2. Parallel processing; 3. Large memory/I/O bandwidth; 4. Reliability; 5. Security	MRI processing benefits from multiple processors, large memory and fast storage. It is not subject to the monitor's 10 ms alarm deadline.
Partner-hospital link	1. Security; 2. Reliable delivery; 3. Data consistency; 4. Availability; 5. Scalable throughput	Records cross organizational boundaries, so authentication, encryption, audit trails and controlled synchronization are required.

1.2 Conflicts and patient-safety priorities

- Low-latency alarms can conflict with heavy background processing. Monitor tasks must receive deterministic priority and must not compete with non-critical workloads.

- High throughput and many concurrent users can increase scheduling and I/O contention on a shared server. Database workloads need capacity planning and isolation.
- Strong encryption and authentication add processing overhead, but this cost is justified for patient confidentiality. Lightweight, hardware-supported security should be used on constrained devices.
- Battery-saving sleep modes can delay communication or sensor processing. A monitor must not enter a power state that violates its alarm deadline.
- The most safety-critical functions are sensor acquisition, alarm detection, alarm delivery, correct patient identification, and reliable access to current records.

1.3 Lessons from OS history

- Batch systems demonstrate efficient processing of large jobs; they are appropriate for scheduled backups and non-urgent image-processing queues, not immediate alarms.
- Time-sharing supports many interactive users and is suitable for desktop and database access, provided resource limits prevent one workload from starving another.
- Personal-computer operating systems provide interactive interfaces and broad application support for doctors' workstations.
- Mobile operating systems provide power management, wireless connectivity, sandboxing and touch interfaces for nurses' tablets.

1.4 Alternative: one general-purpose OS everywhere

A general-purpose OS may provide good average response time, but ordinary scheduling generally does not guarantee a strict worst-case deadline. Under overload, page faults, driver delays or competing tasks, alarm scheduling delay may be unbounded relative to the required 10 ms unless a real-time configuration and verified bounds are provided. A real-time OS is therefore the safer choice for bedside monitors.

A full desktop OS also requires more RAM, storage and background services than a small embedded OS. This increases hardware cost and attack surface on monitors. Central administration might appear simpler because fewer OS types are used, but patching, device drivers, power settings and real-time tuning would be harder to standardize across very different devices. The apparent reduction in OS variety does not outweigh safety and efficiency risks.

Complexity: for n devices, fleet administration effort is approximately O(n) for either approach; however, a single OS does not remove per-device testing and configuration. Memory use is O(1) per device but the constant footprint is substantially larger for a desktop OS than for an embedded RTOS.

TASK 2 – Judge the Type of Operating System for Each Environment

2.1 Recommended OS type

Environment	Choice	Reason
Patient monitors	Embedded hard real-time OS	A hard real-time design is appropriate because the 10 ms alarm deadline is safety-critical. A missed deadline is unacceptable.
Nurses' tablets	Handheld/mobile OS	Provides touch UI, wireless networking, power management, application sandboxing and device management.
Doctors' workstations	Desktop/time-sharing OS	Supports interactive clinical applications, user accounts, multitasking and peripheral devices.
Patient-record database	Clustered OS/service with fail-over	Multiple nodes allow service to continue if one node fails. Replication and tested fail-over reduce downtime.
MRI imaging server	Multiprocessor OS; preferably SMP	Symmetric multiprocessing allows processors to share work and memory, improving parallel throughput for image reconstruction.
Partner hospitals	Distributed system	Independent hospital systems exchange authenticated records over a network while retaining local operation.

2.2 Comparison of OS types

Mainframe systems are designed for very large centralized workloads and high I/O throughput; they could support enterprise-scale transaction processing, but a conventional mainframe is not required for every hospital endpoint. Desktop systems suit interactive workstations. Multiprocessor systems suit compute-intensive MRI processing. Distributed systems connect independent hospital sites. Clustered systems provide coordinated service and fail-over. Hard real-time systems guarantee deadlines under specified conditions; soft real-time systems prioritize timely results but tolerate occasional misses. Handheld systems optimize mobile interaction and battery use.

For the MRI server, symmetric multiprocessing (SMP) is preferred because processors are peers and the OS can schedule parallel tasks across available CPUs. Asymmetric multiprocessing assigns specialized work to particular processors; it can be useful for fixed roles but is less flexible for changing imaging workloads.

2.3 Alternative: one large multiprocessor database machine

A single large multiprocessor database server can offer high throughput and shared-memory communication, but it remains a single point of failure. A motherboard, power, storage-controller or OS failure can stop the service. A cluster of smaller servers with replicated data and automatic fail-over can continue service within seconds, although fail-over is not instantaneous and requires consistency checks.

Cost depends on hardware, licensing, support and redundancy. A large machine may have a high initial cost and expensive upgrades; a cluster adds networking and replication costs but can scale incrementally and avoid total service loss from one node failure. With p processors, ideal compute throughput may approach O(p), but real speedup is limited by serial work, memory bandwidth, locking and

I/O contention (Amdahl’s law). Memory capacity is the sum of installed RAM in a single machine; in a cluster, memory is distributed and is not automatically one shared address space.

TASK 3 – Evaluate Operating-System Structures

3.1 Structure comparison

Structure	Characteristics	Hospital evaluation
Monolithic/simple	Most OS services run in kernel space; fast internal calls.	A faulty driver or service can crash or corrupt the whole kernel; updates may be risky.
Layered	OS is divided into levels with defined interfaces.	Easier to reason about and maintain, but extra layer crossings can add overhead.
Microkernel	Only essential functions remain in kernel; other services run as user-space servers.	Better fault isolation and smaller trusted kernel; message passing can add latency.
Modular	Core kernel supports loadable modules such as drivers and file systems.	Flexible updates and good performance, but a faulty kernel module can still damage the kernel.
Hybrid	Combines microkernel ideas with substantial services in kernel for performance.	Balances compatibility and speed, but is more complex than a minimal microkernel.

3.2 Structure choices

Patient monitors: choose a small microkernel-style or purpose-built RTOS structure. Keep scheduling, interrupt handling and essential IPC in the kernel; isolate non-critical drivers and services where the RTOS supports it. This reduces the amount of code that can compromise the alarm function.

Database server: choose a modular Linux-style kernel with mature storage, networking and monitoring support, deployed as a clustered service. Linux is generally described as a monolithic but modular kernel. Windows uses a hybrid kernel architecture. Android is built on the Linux kernel with a layered application framework and hardware abstraction. QNX is a microkernel RTOS; FreeRTOS is a small real-time kernel, commonly used in embedded systems.

3.3 Alternative: monolithic kernel on monitors

A monolithic kernel can make service requests through a direct system-call path with relatively low overhead. A microkernel may require IPC/message passing between user-space servers, adding context switches and communication cost. However, a direct call is not the only cost: interrupt handling, driver work and scheduling also matter.

A monolithic kernel may use less IPC overhead but includes more services and drivers in privileged memory. A microkernel can keep the kernel smaller, although separate servers still consume memory. Most importantly, a faulty monolithic driver can crash the whole system; an isolated user-space driver/server may fail without taking down the core kernel. For a patient monitor, fault containment is more important than a small reduction in request latency, provided the chosen RTOS still meets the 10 ms deadline.

TASK 4 – Evaluate the Services an Operating System Must Provide

4.1 Service allocation

Environment	Essential services	Services to minimize / controls
Monitors	Program execution, sensor I/O, alarm communication, basic file/log storage, error detection, deterministic resource allocation, authentication and integrity protection.	Remove desktop UI, general-purpose printing, unnecessary multimedia, broad user services and unused network daemons. Retain essential logging and secure updates.
Tablets	Touch UI, application execution, wireless I/O, local encrypted storage, communication, error handling, resource/power management, accounting/audit and strong protection.	Disable unused services and restrict background applications; retain mobile device management and security controls.
Workstations	Full UI, program execution, file and network services, printer/display I/O, error detection, resource allocation, user accounting, access control and endpoint security.	Remove unapproved applications and services; retain patching, logging and endpoint protection.
Database	Program execution, transaction/file services, storage and network I/O, communication, error detection/recovery, resource allocation, accounting, authentication, authorization, encryption and auditing.	Do not remove core file, network, recovery, access-control or monitoring services. Disable only unnecessary packages and interfaces.
MRI server	Compute execution, high-speed storage/network I/O, large-file systems, parallel resource allocation, job accounting, error detection, access control and secure communication.	Avoid unnecessary desktop/background services on the compute host; retain drivers and monitoring needed by the scanner and storage.
Partner link	Network communication, authentication, encryption, authorization, logging, error detection, data validation and controlled file/API access.	No unrestricted shared folders or anonymous access.

4.2 Protecting patient records

Each user and application must be authenticated. Role-based access control should limit users to the records required for their duties. The OS enforces process isolation and file/database permissions; the database applies patient- and operation-level authorization. Data should be encrypted in transit and at rest, access should be audited, and privileged accounts should use stronger authentication. Backups and tested recovery protect availability and integrity.

4.3 Alternative: full desktop OS on tablets

A full desktop OS would typically require more storage and RAM for services, drivers, background processes and a larger application environment. More startup services can lengthen boot time and consume CPU and battery during a shift. A reduced handheld OS has a smaller footprint and integrated power management. The exact memory, storage and boot-time difference depends on the selected OS and configuration; it should be measured on the actual tablet rather than invented. If S is the additional storage footprint and M the additional RAM footprint, the extra space requirement per tablet is $O(S + M)$ in practical capacity terms. Across T tablets, fleet storage provision grows approximately $O(T \cdot S)$.

TASK 5 – Examine System Calls

5.1 Categories of system calls

- Process control: create, execute, wait for and terminate processes (fork, exec, wait, exit).
- File management: create/open, read, write, seek, close and delete files.
- Device management: request/release devices and perform device I/O or configuration.
- Information maintenance: obtain time, process IDs, system information and attributes.
- Communication: create communication channels and exchange data using pipes, sockets or shared memory.
- Protection: set permissions, identities and access-control information.

5.2 What happens during read()

The application calls the C library wrapper read(fd, buffer, count).

The wrapper places the system-call number and arguments in the platform-defined registers or memory locations, then executes a trap/syscall instruction.

The CPU changes from user mode to kernel mode and transfers control to the system-call handler.

The kernel validates the file descriptor, checks permissions and verifies that the user buffer is accessible.

The kernel obtains data from a file cache or storage device; if necessary, it schedules I/O and may block the process.

The kernel copies available bytes to the user buffer and returns the byte count, zero at end-of-file, or -1 with errno set on error.
The CPU returns to user mode and the library wrapper returns to the application.
Parameters may be passed in registers, through a pointer to a block in memory, or on the user stack, depending on the processor ABI and OS. A system-call interface defines the exact convention.

5.3 strace

strace traces system calls and signals made by a Linux process. For example:

```
gcc -Wall -Wextra -std=c11 copy.c -o copy
strace -c ./copy
strace -e trace=openat,read,write,close ./copy
```

The summary shows call counts and time spent in traced system calls. The exact trace may include startup and library activity in addition to the calls made by copy_file().

5.4 Alternative: C library buffering

fopen(), fread() and fwrite() are library functions, not direct system calls. The C library can buffer data in user space, so many small fread/fwrite operations may be combined into fewer read/write system calls. With a library buffer near 4096 bytes, a file of n bytes requires roughly n/4096 data-transfer calls, plus open/close and possible partial-transfer calls. With direct I/O buffer size b, the number is roughly n/b, plus the final EOF read. The library consumes extra memory for its buffers, usually proportional to the number and size of open streams.

TASK 6 – Evaluate the Cost of System Calls

6.1 Performance impact

An ordinary function call normally changes control flow within user mode. A system call crosses the user/kernel protection boundary, saves/restores processor state, validates arguments and permissions, may acquire kernel locks, and may wait for I/O. Therefore, a system call is generally more expensive than a normal function call.

Larger buffers reduce the number of read() and write() calls for a fixed file size. However, excessively large buffers consume more memory and may not improve performance after storage or cache throughput becomes the bottleneck. Library buffering combines small application operations into larger transfers.

6.2 Completed C program

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <stdint.h>



long copy_file(const char *src, const char *dst, size_t bufsize)


{


    char buf[8192];


    if (bufsize == 0 || bufsize > sizeof(buf)) {
        errno = EINVAL;
        return -1;


    }




    int in = open(src, O_RDONLY);
    if (in < 0) return -1;


    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) {
        close(in);
        return -1;
    }



    long calls = 0;
```

```
    ssize_t n;
    while (1) {
        do {


            n = read(in, buf, bufsize);
            calls++;
        } while (n < 0 && errno == EINTR);




        if (n == 0) break;
        if (n < 0) {
            calls = -1;
            break;
        }




        ssize_t  written  =  0;
        while (written < n) {
            ssize_t w = write(out, buf + written, (size_t)(n - written));
            if (w < 0 && errno == EINTR) continue;
            if (w <= 0) {
                calls = -1;
                break;
            }


            written += w;


        }


        if (calls < 0) break;


    }




    close(in);
    close(out);
    return calls;
}
```

```
int main(void)

{

   FILE *f = fopen("records.dat", "w");
   if (!f) {
      perror("fopen");

      return 1;

   }

   for (int i = 0; i < 10000; i++)
      fputc('A' + i % 26, f);
   fclose(f);




   size_t sizes[] = {1, 512, 4096};
   for (int i = 0; i < 3; i++)
      printf("buffer %4zu bytes: %5ld read() calls\n",

            sizes[i], copy_file("records.dat", "copy.dat", sizes[i]));



   unlink("records.dat");
   unlink("copy.dat");
   return 0;
}
```

The program counts the EOF read as a read() call. For a 10,000-byte file, the assignment's expected counts are:

Buffer size	**Expected read() calls**	**Reason**
1 byte	10001	10,000 successful one-byte reads plus one EOF read.
512 bytes	21	20 data reads plus one EOF read.
4096 bytes	4	Three data reads (4096 + 4096 + 1808) plus one EOF read.

6.3 Compile and execute

```
gcc -std=c11 -Wall -Wextra copy.c -o copy

./copy

strace -c ./copy

/usr/bin/time -v ./copy
```

The strace summary may show additional read() calls from program startup and the C library. Compare the total trace count with the program's own copy_file() count, and explain the difference. For a clean comparison, trace only the relevant file descriptors or isolate the copy routine in a minimal test.

6.4 Alternative: memory-mapped files (mmap)

With mmap(), the program maps file contents into its virtual address space and accesses them through memory. It can avoid repeated explicit read() calls and explicit user-buffer copies; mapping and unmapping require a small number of system calls. The OS still handles page faults and storage I/O behind the scenes, so mmap does not eliminate all kernel activity.

For a very large file, mapping the whole file may reserve a large virtual address range and pages become resident as accessed. A windowed mapping limits active memory use. read()/write() with buffer b makes approximately O(n/b) transfer calls; mmap setup is O(1) mapping calls for a single mapping, but page faults are data-dependent. mmap is attractive for random access and large files, while buffered read/write offers explicit control and predictable streaming behavior.

TASK 7 – Evaluate Process Control System Calls

7.1 Process-control operations

- fork() creates a child process. The child initially has a logical copy of the parent's address space, commonly implemented with copy-on-write.
- exec() replaces the current process image with a new program; the process ID remains the same.
- wait()/waitpid() lets a parent collect a child's termination status and release its process-table resources.
- exit() terminates a process and makes its status available to the parent.

A shell parses a command, forks a child, the child uses exec to run the requested program, and the parent waits. The shell obtains the child's status through waitpid() and macros such as WIFEXITED() and WEXITSTATUS(). A zombie is a terminated child whose exit status has not yet been collected; wait() removes it. An orphan is a child whose parent exits first; it is adopted by a system process that can reap it.

7.2 Completed C program

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>


#include <unistd.h>
#include <sys/wait.h>



int main(void)
```

```
{

    int status;
    pid_t pid;


    printf("shell: running job 1\n");
    fflush(stdout);
    pid = fork();



    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {

        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        perror("execlp");
        _exit(127);

    }



    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;

    }

    if (WIFEXITED(status))

        printf("job 1 exit status %d\n", WEXITSTATUS(status));



    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();



    if (pid < 0) {
```

```
        perror("fork");
        return 1;
    }

    if (pid == 0) _exit(3);



    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return 1;

    }

    if (WIFEXITED(status))

        printf("job 2 exit status %d\n", WEXITSTATUS(status));



    return 0;

}
```

Compile and run:

```
gcc -std=c11 -Wall -Wextra process.c -o process

./process
```

Expected output:

```
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

7.3 Why flush stdout before fork?

When stdout is redirected to a file, it is usually fully buffered. If text remains in the parent's user-space output buffer at fork(), both parent and child inherit a copy of that buffer. If both processes later call exit(), each may flush its copy, so the same line can appear twice. fflush(stdout) empties the buffer before fork(). _exit() terminates without flushing inherited stdio buffers; exit() performs normal library cleanup and flushes them.

7.4 Alternative: posix_spawn() or threads

posix_spawn() combines process creation and program execution through a library interface and can be more efficient on some systems, particularly where a complex fork child would be unsafe or expensive. Threads are lighter because they share the same address space and many process resources. However, threads do not provide the same memory isolation: a faulty command running as a thread can corrupt shared memory. For unrelated hospital commands, separate processes provide stronger fault and security isolation.

For 1,000 launches, total time depends on OS, hardware, program size and load. Process creation has per-process page tables and scheduling overhead; fork uses copy-on-write rather than immediately copying all memory. Threads usually use less per-task memory but still require stacks and kernel scheduling structures. Measure rather than assume an exact time.

TASK 8 – Evaluate CPU States and Protection

8.1 CPU modes and protection

User mode restricts applications from executing privileged instructions or directly controlling hardware. Kernel mode allows the OS to manage memory, devices, interrupts and scheduling. A hardware mode bit identifies the current privilege level. A system call or exception transfers control to the kernel through a controlled entry point.

An interrupt is an asynchronous signal from hardware, such as a device requesting service.

A trap/exception is a synchronous event caused by the current instruction, such as a system call or invalid operation.

A timer interrupt returns control to the OS periodically so it can schedule tasks and prevent one process from monopolizing the CPU.

If any user program could disable interrupts, it could prevent the OS from servicing monitor events, scheduling other tasks or responding to faults. A malicious or faulty process could therefore delay alarms and compromise the entire system. Privileged instructions must be restricted to kernel mode.

Memory protection uses virtual address spaces, page tables and hardware permission bits. A process cannot read another process's memory unless the OS deliberately maps shared memory or grants access. Kernel memory is separately protected.

8.2 Alternative: polling instead of interrupts

With polling, the CPU checks a sensor repeatedly. If the polling interval is P, the worst-case detection delay is approximately P plus processing and scheduling delay. Reducing P improves response but increases CPU use and power. With interrupt-driven sensing, the device signals the CPU when an event occurs, so detection delay is closer to interrupt latency plus handler and scheduling time. Interrupts are generally more power-efficient for infrequent events. The monitor should use a verified interrupt-driven design or a bounded hybrid strategy that proves the 10 ms deadline.

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems

9.1 Memory hierarchy

Level	Typical capacity	Typical access time	Hospital use
CPU registers	Bytes to a few KB per core	About 1 CPU cycle	Current instructions and operands.
L1/L2/L3 cache	KB to tens of MB	About 1–tens of ns (varies)	Frequently used code and data.
Main memory (RAM)	GBs to TBs in servers	Tens to ~100 ns	Active database cache, MRI working sets and running programs.
SSD/NVMe	Hundreds of GB to many TB	Typically tens to hundreds of µs for access	Online patient records, recent MRI studies and fast databases.
Hard disk	TBs to tens of TB	Several ms seek/rotational delay	Large archives and backup tiers.
Network storage/cloud	Scalable	Network-dependent: ms or more	Replicated records, shared archives and off-site backup.

Times are approximate and depend on hardware, workload, caching and network conditions; they are intended as a hierarchy comparison, not guaranteed specifications.

9.2 Data placement

- MRI images: active images and processing data in RAM and fast NVMe/SSD; completed studies in redundant high-capacity storage with archival backup.
- Patient records: database working set and indexes in RAM/cache; durable records on redundant SSD storage; encrypted backups and replicas at separate sites.
- Monitor readings: process immediately in monitor memory, send important events to the central system, and retain local buffered records during temporary network outages.

9.3 I/O methods

Programmed I/O makes the CPU repeatedly check and transfer data; it is simple but wastes CPU time for large transfers. Interrupt-driven I/O lets a device notify the CPU when service is needed, reducing constant polling. Direct Memory Access (DMA) lets a controller transfer blocks between a device and memory with limited CPU intervention. DMA is best for high-volume MRI image transfer because it reduces per-byte CPU work. Interrupts can notify the CPU when a DMA block or transfer completes.

9.4 Alternative: one interrupt per word for a 500 MB MRI image

Assuming 500 MB = 500,000,000 bytes and one interrupt per 4-byte word, the transfer would require about 125,000,000 interrupts. If “word” means 8 bytes, it would still require about 62,500,000 interrupts. This creates enormous interrupt-handling overhead and consumes CPU time that could be used for image reconstruction. DMA transfers blocks using a small number of setup/completion events.

Interrupt-per-word handling needs the device and CPU to coordinate each word, often using small registers or buffers. DMA needs a transfer buffer or mapped memory region large enough for the active block(s), plus descriptors; double buffering can overlap transfer and processing. For a large image, DMA is substantially more efficient.

TASK 10 – Measure and Compare

10.1 Measurement procedure

Run all tests on the same Linux machine under similar load. Record the Linux distribution, kernel version, CPU model, RAM, compiler version and whether the storage is SSD or HDD. Repeat each test several times and report the median, because background activity and cache state affect timings.

```
uname -a

cat /etc/os-release
gcc --version
lscpu
free -h

strace -c ./copy

/usr/bin/time -v ./copy
```

10.2 Copy-program results

The assignment supplies the expected read() counts below. They are algorithmic counts for copy_file(), not claimed measured strace totals.

Buffer	Expected read() count	Expected interpretation	Actual strace count/time
1 byte	10001	Many small calls; highest call overhead.	Fill after execution
512 bytes	21	Much fewer calls.	Fill after execution
4096 bytes	4	Fewest calls among tested sizes.	Fill after execution

10.3 User-mode and kernel-mode time

Use /usr/bin/time -v ./copy to observe elapsed time and resource information. The shell time command or /usr/bin/time -p reports real, user and system time. User time is CPU time spent executing application code; system time is CPU time spent in the kernel on behalf of the process. Record the actual values:

Buffer size	Real time	User time	System time	Notes
1 byte	_____	_____	_____	_____
512 bytes	_____	_____	_____	_____
4096 bytes	_____	_____	_____	_____

10.4 Process versus thread creation

A valid comparison can be made with two small C programs: one creates and joins 1,000 children using fork()/waitpid(); the other creates and joins 1,000 pthreads. Use clock_gettime(CLOCK_MONOTONIC, ...) around the creation/join loop and compile the thread program with -pthread. Repeat at least five times and record the median.

gcc -O2 fork_bench.c -o fork_bench

./fork_bench

gcc -O2 -pthread thread_bench.c -o thread_bench

./thread_bench

Method	Time for 1,000	Memory characteristics	Measured result
Processes (fork/wait)	Measure on Linux	Separate virtual address spaces; copy-on-write shares unchanged pages initially; separate process metadata.	_____
Threads (pthread_create/join)	Measure on Linux	Share process code, data and open files; each thread needs its own stack and thread metadata.	_____

Expected trend: threads are often faster and use less memory per task than processes because they share the same process address space. Actual results vary with stack size, kernel, CPU and benchmark design. The measurements support or challenge this trend for the tested machine; they should not be generalized without qualification.

10.5 Alternative: threads instead of processes

If threads are used, creation time is often lower and memory overhead smaller because code, global data and open files are shared. Each thread still needs a stack, registers, scheduling state and kernel bookkeeping. Processes provide separate address spaces and stronger isolation. Thus threads are suitable for cooperating tasks inside one trusted application, while separate processes are safer for

TASK 11 – Recommend and Defend the Operating-System Plan

11.1 Complete recommendation

Environment	OS type/structure	Services and protection
Bedside monitors	Embedded hard RTOS; small microkernel/purpose-built structure	Minimal program execution, sensor I/O, bounded scheduling, alarm communication, secure logging and update protection; interrupt-driven I/O.
Nurse tablets	Managed Android or equivalent handheld OS	Power management, sandboxing, encrypted local data, wireless communication, remote wipe/device management and role-based authentication.
Doctor workstations	Supported desktop Linux or Windows	Full clinical UI, endpoint protection, user accounts, network/file services, audit logging and controlled peripherals.
Database	Linux-based clustered service on redundant nodes	Replication, fail-over, transactional storage, backups, access control, encryption, monitoring and recovery.
MRI server	Linux multiprocessor SMP server	Parallel processing, large RAM, fast NVMe, DMA-capable I/O, job scheduling, secure network storage.
Hospital links	Distributed architecture with secure APIs/VPN or equivalent	Mutual authentication, encryption, least privilege, audit trails, data validation and controlled synchronization.

11.2 Limitations and risks

- A real-time guarantee is only valid for the tested hardware, drivers, task priorities and worst-case workload. The 10 ms deadline must be verified through stress and fault testing.
- A cluster reduces single-node failure but does not eliminate network partition, software defects, operator error or data corruption. Replication and backups must be tested.
- Different OS types increase patching and administration complexity. Central device management, configuration baselines and a documented support lifecycle are required.
- Wireless and partner-hospital links can fail. Monitors should continue local alarm operation and buffer data safely during disconnection.
- Encryption, authentication and logging consume resources. Their overhead must be included in performance and deadline tests.

11.3 Suitability for three hospitals

The plan scales to three hospitals by retaining local real-time monitoring and local clinical operation while connecting the sites through a distributed records service. Each hospital should keep a local cache or operational copy of essential information so a WAN outage does not stop urgent care. Synchronization must resolve conflicts safely and preserve audit history. Shared identity, access policies, encryption standards and incident procedures should be coordinated across all sites.

11.4 Alternative: two large servers with virtual machines

Virtual machines can consolidate workstation, database and imaging workloads and simplify deployment, snapshots and resource allocation. However, placing all services on only two physical servers creates concentrated failure risk. Hypervisor scheduling, virtual I/O and resource contention add overhead. Most importantly, a general-purpose VM on a shared host is not automatically suitable for a hard 10 ms monitor deadline. Bedside monitors should remain on dedicated embedded real-time hardware; critical database services need redundant hosts and storage.

Virtualization consumes memory for guest OS instances and hypervisor structures and consumes CPU for scheduling, emulation and I/O processing. The exact overhead depends on hardware virtualization support, workload and configuration. A two-server design may reduce hardware cost but can worsen reliability if one host failure removes many services. Use virtualization selectively with reserved CPU/memory, high availability and tested recovery—not as a universal replacement.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

Remote home-monitoring devices add untrusted networks, variable latency, intermittent connectivity and a wider range of device capabilities. The original hospital monitors' local real-time requirements remain, but home devices need secure communication, local buffering and an embedded OS suitable for constrained hardware. Cloud records introduce distributed storage, internet-facing APIs, identity federation, encryption, access logging, backup, replication and data-governance requirements.

- Affected OS types: embedded/real-time OS on home devices; handheld/mobile OS for patient-facing applications; distributed and cloud server OS for shared records.
- Affected services: network communication, remote updates, synchronization, authentication, error recovery, logging, storage and resource allocation.
- Affected protection: device identity, mutual authentication, encryption in transit/at rest, least-privilege authorization, secure boot, signed updates and audit trails.

B. Evaluation of the Original Design

The original design remains suitable for the hospital's internal monitors, tablets, workstations, imaging server and local database cluster. It is not sufficient by itself for home monitoring and cloud records because it does not fully address internet exposure, intermittent home connectivity, multi-tenant cloud security or cross-site data synchronization.

A home device cannot rely on a continuous low-latency internet connection for a safety-critical local alarm. It must detect urgent conditions locally and communicate readings when the network is available. Cloud records can improve sharing and scalability, but cloud availability depends on internet access and provider service levels. Strong identity and encryption are necessary because records cross public networks.

C. Adaptation or Redesign

Decision: MODIFY the original design. Retain the real-time hospital monitor architecture and clustered local services, while adding secure edge and cloud components.

- Home monitoring device: use a small embedded OS, with local sensor checks, local alert behavior, secure boot, signed firmware and encrypted local buffering.
- Home-to-cloud communication: use authenticated encrypted channels and a secure gateway/API. Queue readings during network outages and transmit them when connectivity returns.
- Cloud records: use a distributed, highly available database or managed cloud records service with replication, backup, access control, encryption and audit logging.
- Hospital integration: connect each hospital to the cloud through secure APIs; maintain a local cache of essential patient information and define conflict-resolution and synchronization rules.
- Operations: monitor device health, revoke lost/compromised device credentials, patch systems safely and test disaster recovery.

D. Evidence-Based Defence

Modification is preferable to retaining the original plan unchanged because the new requirements introduce internet-connected devices and cloud sharing. A complete redesign would discard working, safety-appropriate local systems unnecessarily. The modified architecture preserves deterministic local alarm processing and the database cluster's availability benefits while adding distributed cloud capabilities.

Trade-offs include additional network and cloud costs, greater administration complexity, dependence on secure identity services and possible synchronization delays. These are managed by keeping immediate alarms local, buffering data during outages, using encrypted authenticated communication, replicating cloud records and maintaining local access to essential information. The decision is supported by the original design priorities: deterministic response for patient safety, fail-over for availability, and distributed services for multi-hospital sharing.

Conclusion

The hospital should use a purpose-built operating-system strategy rather than one general-purpose OS for every device. Hard real-time behavior is essential at the bedside; mobile power management is essential for tablets; desktop systems support clinical workstations; clustered services protect the patient database; and multiprocessor systems accelerate MRI processing. System calls, CPU privilege modes, memory protection, caching and DMA provide the mechanisms needed for safe and efficient operation. The addition of home monitoring and cloud records calls for a controlled modification to a secure distributed architecture while preserving local real-time safety functions.

Appendix – Key Commands and Evidence Checklist

1. Record Linux distribution/version, kernel version, CPU, RAM, compiler version and storage type.
2. Compile the copy program with: gcc -std=c11 -Wall -Wextra copy.c -o copy
3. Run: ./copy ; trace: strace -c ./copy ; timing: /usr/bin/time -v ./copy
4. Compile process-control program with: gcc -std=c11 -Wall -Wextra process.c -o process
5. Run process-control program and compare its output with the expected output in Task 7.
6. Run process and thread benchmarks on the same machine, repeat them, and fill in the measurement tables in Task 10.